

LAGADO — THE MASTER PLAN (Exhaustive, Single Source)

Date: June 3, 2026 · **This replaces all prior plan files.** Archive the rest. Reconciled from:

LAPUTA_v1_0_MASTER_EXECUTION, v4 Unified, VISION.pdf,

FILE_DEPENDENCY_REFERENCE_v3, addenda v1.1–v1.6, and the full working conversation.

Built on the real source files (architecture detail below is from the dependency reference, not memory). Two sources unread/unreadable and NOT incorporated:

LAPUTA_v2_MESH_SCALING_ADDENDUM.pdf (corrupt) and the large duplicate PDFs (sampled). If either holds something to preserve, say so.

THE OPERATING AGREEMENT ("THIS IS THE WAY")

- **The user's will sets direction.** Claude's opinions/judgments are given freely and unfiltered (no yes-man), but they INFORM; they do not STEER. Claude flags risk; the user decides.
 - **The question is never "can it work" — only "how."** Nothing in the user's will is dismissed; if it seems impossible, we find the path, because dismissing it means never finding the how.
 - **Claude stays curious about what the user MEANT** and asks rather than filling the void with its own interpretation.
 - **Build mode = the Haiku task format always** — exact, failsafed, verifiable, never leaving Haiku to interpret vague instructions. That is Claude's job, not Haiku's.
 - **Token-frugal.** Don't re-read what's read; don't regenerate needlessly; flag the wall only when actually near it. The user is broke and will only spend on credits/upgrade if it's continue-or-stop.
 - **We are building a PRODUCT, not a science experiment.** A demo is good; finished is best. ~2 weeks of daily work ahead — build as much as well as we can.
-

PART I — WHAT LAGADO IS (the user's will, stated plainly)

Lagado is the AI interface layer between a person and their computer, for ALL users on ANY OS: you express low-effort intent, Lagado fuses it with your *total private local context*, and executes accurately — so you **do less and are more accurate**. It is a **complete, integrated extension of intent** — chat, agent, assistance across how a person actually uses a computer — not one narrow task. It works **WITH you, then FOR you**: a collaborator that learns your patterns while you stay in control, earning autonomy as it learns you.

Non-negotiables of the user's will (do not override):

1. **For all users. Cross-platform from day one.** The user's machine being Linux is a DEV fact, never a design choice. Everything is built behind OS-agnostic abstractions.
 2. **A complete integrated assistant** (the generality IS the product); a demo is one slice of it.
 3. **Embedded VM is wanted** (not abandoned — see Part VI; the kernel test exists to honor this).
 4. **Immersive is a primary experience / the seller** (literal-desktop vs. lens is the USER's call, left open, not closed by Claude).
 5. **Local, sovereign, private, secure, zero telemetry** — built for people who don't trust AI.
 6. **The user is the first user**; "people like me" (broke, locked out of paid AI, empowered by it) is a legitimate audience, not a dodge.
 7. **Do not psychoanalyze; treat the user's engineering judgment as engineering judgment.**
-

PART II — THE MOAT (why this survives June 2026)

The broad "sovereign local agent" lane is commoditized (Microsoft MXC + OpenClaw Foundation, Hermes, Liquid's own LocalCowork). Lagado does NOT compete there. The lane incumbents are **structurally forbidden** from entering:

- **Intent + your total private LOCAL context → accurate action.**
- Cloud players cannot hold your entire private life locally with nothing leaving — their business model requires your data in their cloud.
- Local-memory players (Rewind, now Meta-owned) only SEARCH context; they don't ACT on it.
- The intersection is open. **The moat and the security guarantee are the same thing:** the only way to safely hold total private context AND act is local + isolated.

The Four Pillars (the differentiator is the combination, esp. Pillar 1):

- **P1 — Maximum Security & Sovereignty (HEADLINE):** isolation by default, encryption at rest, keyring credentials, permission gates on every sensitive action, PII-filtered clipboard, audit log, zero telemetry, offline-capable. Defensible *because incumbents need the cloud*.
- **P2 — Dual-Brain (Hydra-routed):** fast Liquid brain (LFM2.5 family + VL) for interactive/ tool/ vision at sub-second latency; heavy brain on-demand for deep reasoning. **8B/heavy is OPTIONAL, governed by the adaptive resource governor — resource-starved users get a smaller model. AI anywhere.**
- **P3 — Fully Integrated Stack:** one coherent system sharing one substrate, not loose modules.
- **P4 — Persistent Learning (Action Graph):** learns workflows, survives reboots, warm-starts, pre-seeded. (Cross-machine transfer needs a consistent environment — the browser provides it; see V.)

Additive advantages: transformer KV warm-start (<300ms resume on known screens); grid-delta perception (Blake3 per-cell, re-vision only changed regions); cross-modal grounding; optional MoE expert hot-swapping (deferred).

PART III — ARCHITECTURE (detail preserved from the dependency reference)

The InferenceAdapter boundary (already drafted, in mod.rs)

Backend-agnostic trait: `(generate, supports_kv_slots, save/restore/has_kv_slot, model_name, context_size)`. Today: llama-server over HTTP (`(llama_cpp.rs)`). The trait stays generic so backends swap without touching the system. **This is the model of how EVERYTHING cross-platform is built: abstraction + swappable backend.**

llama_cpp.rs — HTTP to llama-server subprocess (port 8080)

Liquid's own LocalCowork uses this pattern (production-validated). Spawns llama-server, waits on `(/health)`, POSTs to `(/v1/chat/completions)`; KV via `(/slots)`; Drop kills the subprocess.

Locked sampling (from Liquid docs): tool routing `(temp=0.1, top_k=50, rep=1.05, max_tokens=512)`; reasoning `(temp=0.2, top_k=80, rep=1.05)`. **NOTE: conversation set max_tokens to 2048 because LFM2.5 is a reasoning model whose CoT exhausted 256 — honor the higher budget on the action path. Tool format (locked from LFM2 training):** tools as a JSON list IN the system prompt text (NOT the chat-template `(tools=)` param — that drops accuracy to ~0%); model emits Pythonic `([tool(arg="x")])`. (Implemented in `(bracket_parser.rs)`; `(parser.rs)` JSON path is superseded/dead.)

hydra.rs — the dual-model orchestrator (the engine's intelligence) [TO BUILD]

LocalCowork's three-step plan→route→synthesize. **Clean context is NON-NEGOTIABLE — history poisoning is the documented cause of a 78%→8% accuracy collapse.** Steps: (1) intent classifier (small model, ~50ms: vision/interactive/reasoning); (2) plan (heavy brain → self-contained steps, bracket format); (3) RAG pre-filter K=15 (operator.rs, by embedding similarity); (4) route (small model picks one tool per step on a CLEAN prompt, zero history); (5) optional verifier (self-consistency N=3-5, escalate thin votes to heavy brain); (6) synthesize (heavy brain streams summary); (7) fall back to single-model loop on any failure.

Perception stack [TO BUILD — cross-platform, see Part V for the browser-first decision]

- `perception/mod.rs` — loop coordinator.
- `capture.rs` — screen capture → /dev/shm @20Hz (native desktop path).
- `delta.rs` — Blake3 per-cell change detection; re-vision only changed regions (<100ms/1000 iters).
- `vlm_adapter.rs` — LFM2-VL via Liquid's shipped SigLIP2 + MLP projector (no training; GUI screens may need fine-tuning later). FALLBACK only.
- `atspi.rs` — semantic accessibility-tree perception (the reliable text backbone). **PRIMARY.**
- `cache.rs` — perception cache in /dev/shm. **Perception principle (settled):** operate at the semantic layer where meaning lives (accessibility tree / DOM / UI-framework), converted to a model-fluent "HTML-like" representation. Vision is fallback for pixel-only apps. STABLE refs keyed on (role+name+path), never a per-call DFS counter (the looping bug). No cleaner layer exists below this (GPU draw-calls/CPU have no semantics).

Living Memory triangle [TO BUILD]

`chronos.rs` (append-only autobiographical timeline — STUB BUILT) $\rightleftharpoons$ `memory_tiers.rs` (thermodynamic heat/decay tiers, /dev/shm Tier 1) $\rightleftharpoons$ `sleep_gate.rs` (consolidation trigger). Every snapshot/summary pairs across all three. Simple form for v1 (remembers corrections, doesn't repeat questions); the thermodynamic/consolidation sophistication is later.

Security layer (THE headline — all to build)

`security/mod.rs` wires: `sandbox.rs` (seccomp+namespaces+cgroups, fail-closed), `crypto.rs` (AES-256-GCM, Argon2id, lazy decrypt, nonce uniqueness), `audit.rs` (append-only tamper-evident log; includes chronos events), `isolation.rs` (capability drops, least privilege), `profile.rs` (tiered Strict/Balanced/Open; sensitive ops ALWAYS gated regardless of profile). **The web-facing agent must be isolated from private context; data crosses only through the human-approved gate; egress enforced (CI egress test), not claimed. Prompt injection is the existential threat — this layer is the answer.**

VM / isolation

`vm/manager.rs` (QEMU supervisor via QMP), `vm/gateway.rs` (hardened Whonix Gateway, owns ALL networking), `vm/network.rs` (workstation has no direct internet), `iso_loader.rs`. See Part VI for engine status.

Capability layer over trust layer (Anthropic harness research)

Planner→Generator→Evaluator (constructs & self-checks a plan) sits on top of Forge bounded-retry + `evaluate_action` gate + HITL + chronos. **Every layer must be removable** as models improve.

Load-bearing connections (memorize)

1. UI/extension $\rightleftharpoons$ main.rs over the **envelope WebSocket** — the only surface $\leftrightarrow$ backend wire.
 2. main.rs $\rightarrow$ llama-server subprocess + orchestration (thin hub).
 3. hydra $\rightarrow$ {liquid, llama_cpp, action_graph, retrieval} — the intelligence.
 4. chronos $\rightleftharpoons$ memory_tiers $\rightleftharpoons$ sleep_gate — living memory; snapshots pair across all three.
 5. security/mod $\rightarrow$ {sandbox, crypto, audit, isolation, profile} — every sensitive path.
 6. vm/manager $\rightarrow$ vm/gateway — workstation never has direct internet.
 7. /dev/shm — shared-memory bus (Tier-1 memory, perception, KV slots, cache).
 8. crypto $\leftarrow$ {memory, auth, vault, mcp, chronos} — encryption substrate.
-

PART IV — CURRENT BUILD STATE (verified, real)

Working / live-verified:

- Agent loop; Liquid-native bracket tool format; Forge bounded retry (3 attempts).
- HITL gate: tiers Read/Write/Destructive; Verdict {Allow, ConfirmTap, ConfirmTyped, Block}; Destructive reserved/unmapped.
- Versioned envelope protocol {v,kind,payload}; uuid id-matched approvals (stale ignored).
- UI permission card across surfaces (ChatProvider single source); live E2E verified.
- chronos append-only log; governor-spawned llama-server (CPU-safe detection).

Carried blockers:

- **Perception NON-FUNCTIONAL** (perceive.py broken path, empty output, refs regenerate $\rightarrow$ agent blind). THE #1 thing the build fixes — via the cross-platform browser path (Part V).
- **libkrun VM** parked pending kernel re-test (Part VI).

Quarantine: obsolete → /obsolete/ (never delete, grep-gated). Only start-llama-server.sh moved. parser.rs/thalamus.py/run_cortex.sh left (referenced). forge.rs NOT obsolete (restored operating basis).

PART V — THE BUILD SURFACE: BROWSER-EXTENSION-FIRST (the user's decision)

The browser is the easiest place to get the loop alive and to pre-seed, AND it is identical on every OS — so it is cross-platform by construction. It is the ON-RAMP, not the end goal. The full-machine integrated assistant is the destination; we start in the browser because the loop comes alive fastest and seeds cleanest there, then expand to native surfaces behind the same abstraction.

Architecture of Backend #1 (browser):

```
Browser EXTENSION (perception of the page DOM + action on the DOM; lives in the
user's REAL browser,
                    real tabs, real logins, real context — "acts WITH you using
YOUR context")
    * envelope WebSocket {v,kind,payload} (ALREADY BUILT — extension is a
new envelope client,
                                         exactly like ChatProvider
connects today)
LOCAL RUST AGENT (the brain: model via governor, Hydra routing, gate, chronos,
memory)
```

- The extension is the eyes/hands IN the browser; the local agent is the brain. Reuses the entire existing envelope protocol + permission gate + chronos — nothing thrown away.
- **Cross-platform from line one:** the extension runs in Chrome/Edge/etc. on any OS; the DOM is the same everywhere; perception/action code is OS-independent.
- **Extension can:** read the active tab's DOM (perception), click/type/act on elements (actuation), see across open tabs, use existing logged-in sessions.
- **Extension cannot (by itself):** run the local model or read local files — that's the local agent's job, over the WebSocket. Clean separation.
- **Perception/action goes behind an abstraction** so native-desktop backends (AT-SPI2 Linux, UIA Windows, AX macOS) add later WITHOUT touching the loop. Browser = Backend #1; native = Backend #2+.

This also resolves portability: a seeded action graph is environment-bound; the browser/DOM is the one environment identical across machines, so browser-seeded graphs travel — which is why "send Koo a copy" becomes feasible later without rebuilding.

PART VI — VM / SANDBOX (honoring "I want the embedded VM")

libkrun status: pending kernel re-test, NOT abandoned. libkrunfw's 6.12 guest hung at first HLT (no timer) under host KVM 7.0.x — confirmed not our code (QEMU+KVM boots 0.4s same host; non-v4 identical; not BORE; both libkrunfw versions hang). **The user is installing `linux-lts` alongside Cachy and we RE-TEST libkrun under it.** BRANCH: boots → libkrun UNBLOCKED, embedded-VM path alive now; still hangs → libkrun parked, file upstream, QEMU/microvm is the backend. **Either way, abstract a `VmBackend` trait** (QEMU now if needed; libkrun behind the trait; HVF/WHPX for Mac/Win — required for cross-platform regardless). The browser loop does NOT depend on the VM; build perception/loop in parallel.

PART VII — THE BUILD ORDER (2 weeks, daily; each step = Haiku tasks with verification gates)

Detailed Haiku tasks are issued per-segment (deep, not shallow). The order:

SEGMENT 0 — Kernel re-test (user + 1 verify task). Install linux-lts alongside; confirm Code-OSS login persists across reboot FIRST; re-test libkrun; record the branch.

SEGMENT 1 — Browser perception+action (Backend #1), cross-platform.

1. Scaffold a `VmBackend`-style perception/action abstraction + the browser extension skeleton.
2. Extension connects to the local agent as a new envelope client (reuse the WS protocol).
3. DOM perception: read the active tab into the model-fluent semantic representation, STABLE refs (role+name+DOM-path), replacing the broken perceive.py for the web surface.
4. DOM actuation: click/type/key on DOM elements by stable ref.
5. Close the loop in-browser: intent → DOM perception → plan → arc-approval (gate) → DOM action → re-perceive → done(). GATE: a trivial web goal COMPLETES (not loops) — first genuinely-working loop.

SEGMENT 2 — Context fusion (the moat). Episodic memory (simple) + current-page perception feeding intent; "remembers my corrections, doesn't repeat questions." Make the private-context advantage real.

SEGMENT 3 — Hydra dual-model routing. plan→route→synthesize with clean-context routing (the 78%→8% guard); small router + heavy/optional reasoning brain; governor decides model tier per hardware.

SEGMENT 4 — Action-graph replay (pre-seed). Verified-success workflows replay WITHOUT the model → fast/confident; the user runs dedicated seeding sessions. (P4; dumb-useful form.)

SEGMENT 5 — Egress proof + security profile. Enforce "nothing private leaves" (CI egress test), make it a VISIBLE moment; wire the tiered security profile (sensitive ops always gated).

SEGMENT 6 — Immersive experience (USER's call on form). The watch-and-direct surface — render the agent's activity legibly (meaning + decision-point snapshots + the approve/deny moment). Literal-desktop vs. lens is the user's decision, left open here by design.

SEGMENT 7 — Native-desktop backend(s) behind the abstraction. AT-SPI2 / UIA / AX as Backend #2+ — the expansion from browser-on-ramp to whole-machine. (Honors "for all users, the whole computer.")

Run rough end-to-end first within each segment, then optimize and test nonstop. Make it work, then good.

PART VIII — THE LADDER (north star, from VISION.pdf — earned, not built first)

- **v1.0 Lagado (app)** — sovereign local agent, dual-brain, browser-on-ramp → whole machine, isolated. NOW.
 - **v1.1 Lagado Mobile** — via Liquid LEAP SDK (iOS/Android); same dual-brain/action-graph/vault/MCP/ tiered security; no Qwen/QEMU on phone (OS-level sandbox); host control via platform accessibility APIs.
 - **v2.0 Lagado OS** — Qubes-derived; Liquid as the inter-Qube intelligence SUBSTRATE (AI lives BETWEEN Qubes, not inside them). EARNED after v1 ships — not built first.
 - **v3.0 The Liquid + Lagado stack** — full local ecosystem. **v1 must prove:** dual-brain viable on consumer hardware; isolation works for a host-acting agent; action-graph + retrieval makes small models punch above weight; Liquid models good enough as default brain; adopt-not-train is the right discipline. **Sleep-distillation (v2 north star, v1 HOOKS ONLY — cheap, build now):** tag action-graph entries with `verified_success`; tag accepted `self_model` statements; keep a replay-data manifest — so the eventual QLoRA training set is assemblable without retrofitting.
-

PART IX — THE DEMO / DISTRIBUTION (build the product; package later)

Build the real product, usable on the user's machine, as well as possible. The "send Koo a copy" packaging is determined LATER, once it's well-made. By then it's a strong thing, not a demo hack. The browser-first surface is what makes "send it, runs on his unknown OS" feasible when the time comes.

Koo Seung-yub, CEO, Wonderful Platform (1thefull.com), Seoul (girlfriend's father): established AI **care-robot** company (Dasom/Dasomi KR, Avadin US; "Physical AI Care OS" CES 2026; 10,000+ robots / 80+ Korean govt welfare-health centers; 70+ patents; KT + OpenAI partners; expanding US/Japan/EU). **Strategic fit:** their thesis ("invisible constant AI that learns the person and quietly helps") IS this thesis. **Their gap = the message:** their robots run on CLOUD GPT → elderly health/life-log data goes to OpenAI's cloud → privacy/compliance liability in HIPAA/GDPR markets they're entering. Lagado's local-sovereign on-device intelligence is the fix. Show the MECHANISM on real data, in English; let a domain-expert extrapolate. **Outcome open** (adoption / venture / advising / connections) — the same artifact serves all; don't pre-optimize or build custom for his robots before commitment.

PART X — BUSINESS / STRUCTURE (deferred by design)

Profit / non-profit / open-source decided AFTER traction, not before. LFM Open License clears proprietary commercial use free to \$10M annual revenue (then a Liquid deal); conditions: attribution

"Powered by Liquid," document modifications, bundle the license, patent-peace. Funding paths if needed:

NLnet/NGI (solo-friendly, no network), the Liquid relationship (you're a showcase), the Wonderful

Platform relationship (highest-leverage). Do not raise; let a working thing pull resources.

PART XI — DEPENDENCIES (approved exceptions; otherwise none without sign-off)

uuid v4 (DONE) · atspi + zbus (native perception, approved — but Backend #2, after browser) · krun-sys (with libkrun, pending kernel test) · sysinfo (governor, when built) · QEMU (system dep, VM

backend). Browser extension: standard web-extension APIs + the existing envelope protocol; no heavy new

Rust deps for the browser surface.

PART XII — KILLED IDEAS (with reasons — do not re-litigate)

- **Build an OS / modify Qubes as v1 base** — that IS maintaining an OS fork solo; unfinishable; OS is v2, earned.
 - **"Origin Pilot / Quantum OS proves it"** — FALSE; it's a quantum-COMPUTER OS, unrelated; the supervisor narrative is unverified/confabulated. Never cite as prior art.
 - **Pure-vision GUI control / "stream video like Tesla FSD"** — relocates the hard problem; vision is fallback, semantic layer primary. "Constant recurrent state / days of video / no KV-cache" is FALSE for LFM2.5 (hybrid, has a cache) — never claim it.
 - **Linux-only anything** — violates "for all users." Cross-platform from line one.
 - **Purchased / Korean training data for the demo** — contradicts the moat (value = REAL private context). No bought data; Korean is a post-finish feature.
 - **SaaS / cloud agent** — contradicts the local moat; recurring cost the user can't afford.
 - **"Un-hackable / military-grade / physically impossible"** in the pitch — absolutes are a dare in security; say "strong isolation / sandboxed by design."
-

PART XIII — OPEN QUESTIONS (honestly flagged, the user's to decide)

- Immersive form: literal-desktop vs. lens — USER's call, deliberately not closed.
- Extension distribution later (store vs. sideload) and how "send Koo a copy" is packaged.
- Kernel test outcome (libkrun unblocked or parked) — determines VM near-path.
- Whether the corrupt mesh addendum / unread PDFs hold anything to preserve.
- Profit/non-profit/open structure — deferred to post-traction.

— *Lagado Master Plan (exhaustive). All prior plan files may now be archived.*